



RELATÓRIO DE DESENVOLVIMENTO

Desenvolvimento de uma Plataforma SaaS de Observabilidade para Aplicações Web

Parte prática do Trabalho de Conclusão de Curso em Ciência da Computação

FOCO DO MVP	Tracing com OpenTelemetry
SITUAÇÃO	Fundação, ingestão, SDK Node e demonstração implementados
PRÓXIMA ETAPA	Consultas e visualização de traces no dashboard

Documento preparado para acompanhamento acadêmico e orientação

01

Resumo executivo

O projeto tem como objetivo desenvolver uma plataforma SaaS capaz de receber, processar, armazenar e apresentar telemetria de aplicações web. O padrão adotado é o OpenTelemetry, com prioridade para tracing no MVP. A pergunta central que orienta a solução é: onde a aplicação está apresentando problemas e em qual parte da execução o tempo está sendo gasto?

ETAPAS CONCLUÍDAS

5

Arquitetura até aplicação demonstrativa

TESTES APROVADOS

15

API, ingestão, segurança, SDK e demo

SINAL PRIORITÁRIO

Tracing

Metrics e logs preparados, não persistidos

Entregas consolidadas

- Monorepo TypeScript com aplicações Next.js e NestJS, pacotes compartilhados e padronização de qualidade.
- Modelagem PostgreSQL/Prisma para usuários, projetos, credenciais, ambientes, serviços, traces e spans.
- Autenticação JWT, refresh token rotativo, cookies HttpOnly e senhas protegidas com Argon2id.
- Gestão de projetos, environments e Project Keys armazenadas somente por hash/HMAC.
- Recepção OTLP HTTP/gRPC no Collector e ingestão OTLP/gRPC na API NestJS.
- Sanitização de telemetria, controle de cardinalidade, descoberta de serviços e persistência idempotente.
- SDK Node simplificado com auto-instrumentação e API para spans manuais de domínio.
- Aplicação Next.js demonstrativa com cenários controlados de latência, erro, banco e operação composta.
- Infraestrutura Docker Compose, Nginx, bancos separados e documentação técnica para o TCC.

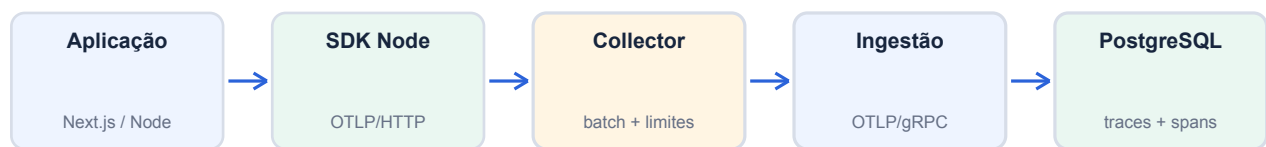
Estado atual: a plataforma já possui o caminho de escrita da telemetria. A próxima etapa implementa o caminho de leitura, agregação e visualização no dashboard.

02

Arquitetura implementada

A arquitetura separa instrumentação, transporte, ingestão, processamento, persistência e apresentação. Essa divisão evita concentrar regras de negócio no Next.js e mantém os endpoints de alta frequência isolados da API administrativa.

FLUXO DE ESCRITA



FLUXO DE LEITURA E ADMINISTRAÇÃO

Componente	Responsabilidade implementada
Dashboard Next.js	Fundação da interface web e preparação para páginas administrativas e de tracing.
API NestJS	Autenticação, projetos, environments, chaves, health check e ingestão OTLP/gRPC.
OpenTelemetry Collector	Recepção OTLP, limite de memória, batching por projeto e encaminhamento seguro.
PostgreSQL / Prisma	Dados administrativos, serviços descobertos, traces e spans.
SDK Node	Configuração simplificada do OpenTelemetry e exportação OTLP/HTTP.
Aplicação demo	Geração reproduzível de traces normais, lentos, com erro e acesso ao banco.
Nginx	Ponto de entrada da plataforma, proxy reverso e limites de requisição.

Decisões centrais

- A identidade de um serviço usa projeto, environment e service.name; não depende da URL.
- OTLP permanece o protocolo entre aplicações, Collector e ingestão; não foi criado protocolo proprietário.
- W3C Trace Context foi definido como mecanismo de correlação entre browser e backend.
- Tracing é a prioridade. Metrics e logs possuem pipelines de diagnóstico, sem persistência prematura.

03

Domínio, banco de dados e autenticação

A modelagem foi preparada para múltiplos projetos, ambientes e serviços por usuário. O mesmo projeto pode reunir frontend, backend e outros serviços, mesmo quando todos usam o mesmo domínio.

Entidade	Finalidade	Proteções e observações
User	Conta da pessoa usuária	Senha armazenada com Argon2id.
AuthSession	Sessões renováveis	Refresh token persistido somente por hash.
Project	Aplicação ou produto monitorado	Todas as consultas validam ownership.
ProjectKey	Autenticação da ingestão server-side	Segredo exibido uma vez; banco armazena HMAC-SHA-256.
ProjectAllowedOrigin	Origens autorizadas para o futuro SDK browser	Preparação para CORS, quotas e identificador público.
Environment	production, staging ou development	Escopo obrigatório para serviços e traces.
Service	Processo lógico instrumentado	Descoberto por service.name e atributos OpenTelemetry.
Trace	Resumo de uma operação distribuída	Mantém duração, status, início, fim e serviço raiz.
Span	Unidade individual de trabalho	Atributos, eventos e links sanitizados em JSONB.

Funcionalidades administrativas implementadas

Área	Operações disponíveis
Autenticação	Cadastro, login, consulta de sessão, renovação e logout.
Projetos	Criação, listagem e consulta individual com validação de proprietário.
Environments	Criação e listagem por projeto.
Project Keys	Geração, exibição única, listagem de prefixos, rotação e revogação.

Índices foram definidos especialmente para `traceId`, `spanId`, `projectId`, `serviceId`, `startedAt`, `durationMs` e `statusCode`. A modelagem também prevê consultas temporais e cálculo correto de percentis no PostgreSQL.

04

Ingestão, processamento e segurança

O Collector recebe telemetria das aplicações e encaminha os traces para uma porta interna dedicada da API. A ingestão não compartilha controllers com os endpoints normais do dashboard.

Etapa	Implementação
1. Recepção	OTLP/gRPC em 4317 e OTLP/HTTP em 4318 no Collector.
2. Controle	memory_limiter e batches de até 1.024 itens no Collector.
3. Isolamento	Batches separados pelo metadata x-obs-project-key.
4. Encaminhamento	OTLP/gRPC para o serviço interno NestJS na porta 4319.
5. Autenticação	Validação de formato, hash/HMAC, revogação e projeto proprietário.
6. Processamento	Conversão OTLP, normalização, sanitização e controle de cardinalidade.
7. Persistência	Descoberta de services e gravação idempotente de traces e spans.

Controles de segurança implementados

Controle	Comportamento
Credenciais	Authorization, cookies, senhas, tokens, secrets e connection strings são descartados.
URLs	Query strings e fragmentos são removidos; IDs numéricos e UUIDs viram :id.
Payload	Máximo de 2.000 spans por exportação e limite de 4 MiB no transporte.
Atributos	Até 64 por objeto, arrays de 32 itens, profundidade 4 e strings de 2.048 caracteres.
Eventos e links	Até 32 itens por span, sempre sujeitos à sanitização.
Banco	Parâmetros SQL, credenciais e valores de conexão não são coletados pelo SDK.
Segregação	Project Key permanece no metadata e nunca é transformada em atributo de span.

A persistência suporta reenvio de lotes e chegada fora de ordem. Spans são deduplicados, enquanto o resumo do trace é atualizado conforme novos limites temporais ou o root span são recebidos.

05

SDK Node e aplicação demonstrativa

Foi criado o pacote `@tcc-observability/node` para reduzir a instrumentação de uma aplicação Node.js a uma configuração pequena e explícita.

```
import { initObservability } from
 '@tcc-observability/node';

initObservability({
  projectKey: process.env.OBS_PROJECT_KEY!,
  serviceName: 'envcove-server',
  environment: 'production'
});
```

Capacidade	Situação
Exporter OTLP/HTTP protobuf	Implementado com Project Key no header server-side.
Auto-instrumentação	HTTP, fetch/Undici, frameworks suportados e PostgreSQL.
Spans manuais	Função <code>traceOperation</code> para etapas relevantes do domínio.
Validação	Recusa chave inválida, endpoint não HTTP e credenciais embutidas na URL.
Ciclo de vida	Inicialização idempotente e shutdown gracioso.
Next.js	Carregamento pelo hook <code>instrumentation.ts</code> apenas no runtime Node.

Aplicação demonstrativa

A aplicação TraceLab foi criada para tornar a apresentação reproduzível. A interface permite disparar cada comportamento e conferir status, duração e resposta antes de abrir o trace correspondente na plataforma.

Endpoint	Cenário	Evidência esperada
GET /api/fast	Resposta imediata	Referência de baixa latência.
GET /api/slow	Atraso intencional de 900 ms	Operação lenta e impacto nos percentis.
GET /api/error	Exceção controlada e HTTP 500	Trace e span com status de erro.
GET /api/database	Consulta PostgreSQL	Span automático do driver pg.
POST /api/order	Validação, estoque, pagamento e persistência	Waterfall de spans manuais e banco.

06

Infraestrutura, documentação e qualidade

O Docker Compose foi preparado para reproduzir toda a topologia local e reduzir diferenças entre máquinas de desenvolvimento. A infraestrutura é intencionalmente simples, adequada ao escopo acadêmico do MVP.

Serviço	Papel
postgres	Banco administrativo e de telemetria da plataforma.
demo-postgres	Banco isolado usado pela aplicação demonstrativa.
migrate	Aplicação automática das migrations antes de liberar a API.
api	NestJS REST em 4000 e ingestão OTLP/gRPC interna em 4319.
web	Dashboard Next.js em 3000.
otel-collector	OTLP/gRPC 4317, OTLP/HTTP 4318 e health 13133.
demo	Aplicação de demonstração em 3100.
nginx	Entrada da plataforma na porta 80.

Documentação produzida

Documento	Conteúdo
README.md	Objetivo, instalação, variáveis, Docker, comandos, instrumentação e testes.
docs/architecture.md	Componentes, fronteiras, decisões e evolução incremental.
docs/database.md	Entidades, relacionamentos, índices, JSONB e percentis.
docs/telemetry-flow.md	Fluxo Node, contrato Collector-ingestão, correlação e reenvio.
docs/instrumentation.md	SDK Node, Next.js, propagação e cardinalidade.
docs/security.md	Credenciais, sanitização, browser e logging interno.
apps/demo/README.md	Preparação e execução dos cenários de demonstração.
packages/node/README.md	API pública, spans manuais e garantias do SDK.

Validação da etapa

LINT Aprovado Todos os workspaces	TYPECHECK Aprovado Sem erros TypeScript	TESTES 15 aprovados 9 API + 3 SDK + 3 demo
---	---	--

07

Limitações atuais e próximos passos

O caminho de ingestão e persistência está implementado, mas o MVP ainda precisa do caminho de consulta e das visualizações operacionais. As limitações abaixo foram mantidas explícitas para evitar apresentar como concluído algo que ainda depende de implementação ou validação externa.

Limitação atual	Impacto
Docker indisponível no ambiente de desenvolvimento utilizado	Os arquivos foram preparados, porém o fluxo completo em containers ainda não foi executado localmente.
Sem banco/Collector reais durante a validação final	Testes unitários e builds passaram, mas o teste ponta a ponta OTLP-Collector-API-PostgreSQL permanece pendente.
Dashboard de tracing ainda não implementado	Os traces podem ser ingeridos e persistidos, mas ainda não possuem listagem, filtros ou waterfall na interface.
Browser SDK ainda não iniciado	A correlação frontend-backend está desenhada, porém pertence à prioridade 2.
Retenção ainda sem rotina executável	O schema e a configuração existem; falta o job de exclusão de dados antigos.

Plano das próximas etapas

Ordem	Etapa	Resultado esperado
1	API de consultas	Services, traces, operações, erros e agregações temporais.
2	Overview	Requests, error rate, média, p50, p95, p99 e séries temporais.
3	Lista de traces	Filtros por período, ambiente, serviço, status, duração e operação.
4	Waterfall	Relação pai-filho, timeline, serviço, duração, atributos e eventos.
5	Performance e erros	Operações lentas, serviços lentos, traces lentos e agrupamento de falhas.
6	Retenção e hardening	Exclusão periódica, rate limiting e testes integrados.
7	Browser SDK	Page loads, fetch/XHR, erros frontend e correlação W3C.
8	Funcionalidades secundárias	Uptime, alertas, metrics e logs conforme tempo disponível.

Conclusão parcial

A base técnica necessária para demonstrar tracing está construída: existe um modelo de dados coerente, ingestão baseada em padrões oficiais, proteção de credenciais, um SDK de integração simples e uma aplicação capaz de gerar cenários previsíveis. O próximo marco transforma os dados armazenados em informação útil por meio das consultas e visualizações do dashboard.